

Financial Fraud Detection Project

API Routes, Request/Response Samples, Rule Engine, ML, ROC, and Dashboard Usage

Generated from the current server-changed branch on 2026-05-13

This PDF is built from the live project files in E:\CAPSTONE and includes sample payloads for Express and Flask routes.

Financial Fraud Detection Project API and Logic Guide

Generated from the current server - changed branch on 2026-05-13.

This guide explains the active Express routes, Flask ML routes, request bodies, response shapes, dashboard usage, rule-based fraud logic, ML scoring, ROC/FPR/TPR, and how the frontend should use response data.

1. Project Overview

The project has two backend services.

Service	Location	Default Port	Purpose
Express API	server	4000	Authentication, profile, transactions, fraud scoring, analytics, admin dashboard APIs.
Flask ML API	ml-service	5000	Trains and serves a fraud probability model using scikit-learn.

Main Express route mounts:

Base Path	File	Purpose
/api/v1/auth	server/routes/User.js	Signup, login, OTP, password operations.
/api/v1/profile	server/routes/Profile.js	User profile CRUD for current logged-in user.
/api/v1/transaction	server/routes/Transaction.js	Create and list transactions.
/api/v1/analytics	server/routes/Analytics.js	User/admin dashboard summary.
/api/v1/admin	server/routes/Admin.js	Admin-only fraud dashboard and user drill-down APIs.

2. Authentication Pattern

Protected routes use the auth middleware. Send the JWT token in one of these places:

Authorization: Bearer <jwt_token>

OR:

Cookie: token=<jwt_token>

or, less preferred:

```
{
  "token": "<jwt_token>"
}
```

Role middleware:

Middleware	Allows
auth	Any logged-in, non-blocked user.
isUser	accountType === "User"
isAdmin	accountType === "Admin"

Common auth errors:

```
{ "success": false, "message": "Token missing" }

{ "success": false, "message": "Invalid token" }

{ "success": false, "message": "Admin access only" }

{ "success": false, "message": "Account is blocked" }
```

Frontend use:

- If 401, clear auth state and send user to login.
- If 403, show role/access/account-blocked message.
- Use `data.accountType` from login response to decide whether to show user dashboard or admin dashboard.

3. Express API Route Summary

Method	Route	Auth	Purpose
GET	/	No	Health check for Express server.
POST	/api/v1/auth/sendotp	No	Send signup OTP.
POST	/api/v1/auth/signup	No	Register user using OTP.
POST	/api/v1/auth/login	No	Login and return JWT plus full user details.
POST	/api/v1/auth/changepassword	Yes	Change logged-in user's password.
POST	/api/v1/auth/reset-password-token	No	Generate reset-password token.
POST	/api/v1/auth/reset-password	No	Reset password using token.
POST	/api/v1/profile/updateprofile	Yes	Create/update current user's profile.
GET	/api/v1/profile/me	Yes	Get current user's full profile.
POST	/api/v1/profile/editprofile	Yes	Edit current user's user/profile fields.
POST	/api/v1/transaction/create-transaction	User	Create transaction and run fraud scoring.
GET	/api/v1/transaction/get-transactions	User	Get current user's transactions.
GET	/api/v1/transaction/get-all-transactions	Admin	Get paginated transaction list.
GET	/api/v1/analytics/get-analytics	Yes	Dashboard summary; admin sees all, user sees own.
GET	/api/v1/analytics/admin	Admin	Same dashboard summary but admin-protected.
GET	/api/v1/admin/overview	Admin	Admin fraud dashboard with graphs, reasons, top users.
GET	/api/v1/admin/users	Admin	Paginated users with fraud/risk summary.
GET	/api/v1/admin/users/:userId	Admin	User drill-down for clicked user.
GET	/api/v1/admin/scoring-guide	Admin	Numeric scoring guide for frontend labels and graphs.

4. Base Server Route

GET /

Purpose: check if the Express server is running.

Response:

```
{
  "success": true,
  "message": "Server is running "
}
```

5. Auth Routes

POST /api/v1/auth/sendotp

Purpose: sends signup OTP to email. It rejects an email that already belongs to a user.

Request:

```
{
  "email": "raushan@example.com"
}
```

```
}
```

Success response:

```
{
  "success": true,
  "message": "OTP sent successfully",
  "otp": "483921"
}
```

Possible errors:

```
{ "success": false, "message": "Email required" }

{ "success": false, "message": "User already exists" }
```

Frontend use:

- Call before signup.
- In production, do not show OTP in response. Keep OTP email-only.

POST /api/v1/auth/signup

Purpose: create a new user after OTP verification.

Request:

```
{
  "firstName": "Raushan",
  "lastName": "Kumar",
  "email": "raushan@example.com",
  "password": "Pass@12345",
  "confirmPassword": "Pass@12345",
  "otp": "483921"
}
```

Success response:

```
{
  "success": true,
  "redirectTo": "update-profile"
}
```

Important behavior:

- accountType defaults to "User".
- Server creates a default avatar URL.
- Server sets an HTTP-only token cookie.
- Response does not return the token directly on signup.

Frontend use:

- On success, navigate to profile completion page.

POST /api/v1/auth/login

Purpose: authenticate user and return JWT plus user/profile data.

Request:

```
{
  "email": "raushan@example.com",
  "password": "Pass@12345"
}
```

Success response:

```
{
```

```

"success": true,
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",
"data": {
  "_id": "665f10000000000000000001",
  "firstName": "Raushan",
  "lastName": "Kumar",
  "email": "raushan@example.com",
  "accountType": "User",
  "isBlocked": false,
  "behavior": {
    "avgAmount": 2200,
    "stdAmount": 850,
    "maxAmount": 6000,
    "transactionCount": 12,
    "commonDevices": ["dev-android-1"],
    "commonLocations": ["Patna"],
    "lastTransactionAt": "2026-05-13T07:30:00.000Z",
    "lastLatitude": 25.5941,
    "lastLongitude": 85.1376
  },
  "profile": {
    "dob": "2002-05-20T00:00:00.000Z",
    "gender": "Male",
    "address": {
      "city": "Patna",
      "state": "Bihar",
      "country": "India"
    }
  }
}
}

```

Frontend use:

- Store token.
- Send it on protected routes as Authorization: Bearer <token>.
- Use data.accountType:
- "User" -> user dashboard. - "Admin" -> admin dashboard.

POST /api/v1/auth/changedpassword

Purpose: change password for logged-in user.

Headers:

```
Authorization: Bearer <token>
```

Request:

```

{
  "oldPassword": "Pass@12345",
  "newPassword": "NewPass@12345",
  "confirmPassword": "NewPass@12345"
}

```

Success response:

```

{
  "success": true,
  "message": "Password changed successfully. Please login again."
}

```

Frontend use:

- Clear auth state.
- Redirect user to login.

POST /api/v1/auth/reset-password-token

Purpose: create a reset token for a user email.

Request:

```
{
```

```
    "email": "raushan@example.com"
  }
```

Success response:

```
{
  "success": true,
  "message": "Reset password email sent successfully",
  "data": {
    "token": "8ec8f101-1111-4444-aaaa-123456789000",
    "url": "http://localhost:3000/reset-password/8ec8f101-1111-4444-aaaa-123456789000"
  }
}
```

Important behavior:

- Token expires after 5 minutes.
- Mail sending is currently commented in `ResetPassword.js`, so the token is returned in response for development.

POST /api/v1/auth/reset-password

Purpose: reset password using token.

Request:

```
{
  "token": "8ec8f101-1111-4444-aaaa-123456789000",
  "password": "NewPass@12345",
  "confirmPassword": "NewPass@12345"
}
```

Success response:

```
{
  "success": true,
  "message": "Password reset successful"
}
```

6. Profile Routes

POST /api/v1/profile/updateprofile

Purpose: save profile after signup. `address.city` is required.

Headers:

```
Authorization: Bearer <token>
```

Request:

```
{
  "dob": "2002-05-20",
  "gender": "Male",
  "address": {
    "current": "Kankarbagh",
    "permanent": "Patna",
    "city": "Patna",
    "state": "Bihar",
    "country": "India",
    "pincode": "800020"
  }
}
```

Success response:

```
{
  "success": true,
  "message": "Profile updated",
  "redirectTo": "dashboard",
  "data": {
    "_id": "665f10000000000000000001",
    "firstName": "Raushan",
    "lastName": "Kumar",
    "email": "raushan@example.com",
    "profile": {

```

```

    "dob": "2002-05-20T00:00:00.000Z",
    "gender": "Male",
    "address": {
      "current": "Kankarbagh",
      "permanent": "Patna",
      "city": "Patna",
      "state": "Bihar",
      "country": "India",
      "pincode": "800020"
    }
  }
}

```

Frontend use:

- Save returned data in user state.
- Navigate to dashboard.

GET /api/v1/profile/me

Purpose: get current logged-in user's full user/profile object.

Headers:

```
Authorization: Bearer <token>
```

Success response:

```

{
  "success": true,
  "data": {
    "_id": "665f10000000000000000001",
    "firstName": "Raushan",
    "lastName": "Kumar",
    "email": "raushan@example.com",
    "accountType": "User",
    "profile": {
      "gender": "Male",
      "address": {
        "city": "Patna",
        "state": "Bihar"
      }
    }
  }
}

```

POST /api/v1/profile/editprofile

Purpose: edit current user's user fields and profile fields.

Request:

```

{
  "firstName": "Raushan",
  "lastName": "Raj",
  "image": "https://res.cloudinary.com/demo/profile.png",
  "dob": "2002-05-20",
  "gender": "Male",
  "address": {
    "current": "Boring Road",
    "city": "Patna",
    "state": "Bihar",
    "country": "India",
    "pincode": "800001"
  }
}

```

Success response:

```

{
  "success": true,
  "message": "Profile updated",
  "redirectTo": "dashboard",
  "data": {
    "firstName": "Raushan",
    "lastName": "Raj",
    "profile": {
      "address": {
        "city": "Patna",

```

```

    "state": "Bihar"
  }
}
}

```

7. Transaction Routes

POST /api/v1/transaction/create-transaction

Purpose: create a transaction, calculate behavior features, run rule engine, call ML service if available, store final risk scores, and save decision.

Auth:

- Requires logged-in user.
- Requires accountType === "User".

Request:

```

{
  "amount": 2500,
  "merchant": "Amazon",
  "category": "shopping",
  "channel": "mobile",
  "accountId": "primary-account",
  "deviceId": "dev-android-1",
  "location": {
    "latitude": 25.5941,
    "longitude": 85.1376,
    "country": "IN"
  }
}

```

Required fields:

Field	Required	Notes
amount	Yes	Number.
deviceId	Yes	Used for known/new device rule.
location.latitude	Yes	Used for city lookup and distance.
location.longitude	Yes	Used for city lookup and distance.
merchant	No	Defaults to Unknown merchant in schema.
category	No	Defaults to general.
channel	No	One of card, web, mobile, atm; default is card.
accountId	No	Defaults to primary-account.

Normal/approved response sample:

```

{
  "transaction": {
    "_id": "6660aa000000000000000001",
    "user": "665f10000000000000000001",
    "amount": 2500,
    "merchant": "Amazon",
    "category": "shopping",
    "channel": "mobile",
    "accountId": "primary-account",
    "deviceId": "dev-android-1",
    "location": {
      "city": "Patna",
      "country": "IN",
      "latitude": 25.5941,
      "longitude": 85.1376
    }
  },
  "features": {
    "amount": 2500,
    "hour_of_day": 14,
    "velocity_10m": 0,
    "velocity_1h": 1,
    "time_gap_minutes": 90.2,
    "geo_distance_km": 0,
    "device_changed": 0,
  }
}

```



```

    "amount_deviation": 0.42,
    "night_transaction": 0,
    "known_device": 1,
    "known_location": 1
  },
  "scores": {
    "ml": 0.03,
    "rule": 0.12,
    "anomaly": 0,
    "graph": 0.063,
    "risk": 0.063
  },
  "adaptivePolicy": {
    "otpThreshold": 0.5,
    "blockThreshold": 0.7,
    "profileConfidence": 0.5
  },
  "decision": "allow",
  "status": "approved",
  "explanation": {
    "type": "score_based",
    "reason": "Risk below OTP threshold",
    "message": "Decision based on weighted rule, anomaly, and ML scores.",
    "mlMessage": "ML fraud probability 0.03 at threshold 0.5",
    "rules": [
      { "name": "velocity", "score": 0.1, "level": "low", "reason": "Normal movement" },
      { "name": "frequency", "score": 0.1, "level": "low", "reason": "Normal frequency" },
      { "name": "amount", "score": 0.1, "level": "low", "reason": "Normal amount" },
      { "name": "device", "score": 0.1, "level": "low", "reason": "Known device" },
      { "name": "location", "score": 0.1, "level": "low", "reason": "Known location" }
    ]
  }
}
}
}

```

Blocked response sample:

```

{
  "transaction": {
    "_id": "6660aa0000000000000000002",
    "amount": 95000,
    "merchant": "Unknown merchant",
    "deviceId": "new-device-77",
    "location": {
      "city": "Delhi",
      "country": "IN",
      "latitude": 28.6139,
      "longitude": 77.209
    },
    "features": {
      "amount": 95000,
      "hour_of_day": 2,
      "velocity_10m": 7,
      "velocity_1h": 10,
      "time_gap_minutes": 1.2,
      "geo_distance_km": 850,
      "device_changed": 1,
      "amount_deviation": 6.8,
      "night_transaction": 1,
      "known_device": 0,
      "known_location": 0
    },
    "scores": {
      "ml": 0.94,
      "anomaly": 1,
      "rule": 1,
      "graph": 1,
      "risk": 1
    },
    "decision": "block",
    "status": "blocked",
    "explanation": {
      "type": "hard_rule",
      "reason": "Too many transactions in short time",
      "message": "Rule engine made the final decision. ML score is stored for review and graphing.",
      "mlMessage": "ML fraud probability 0.94 at threshold 0.5",
      "rules": [
        { "name": "frequency", "score": 1, "level": "critical", "reason": "Burst transactions" },
        { "name": "device", "score": 0.7, "level": "high", "reason": "New device" }
      ]
    }
  }
}
}

```

Frontend use:

Field	How to use
transaction.decision	Main UI flow switch: allow, otp, block.

Field	How to use
transaction.status	Display state: approved, pending_otp, blocked.
transaction.scores.risk	Risk meter and graph value.
transaction.scores.rule	Rule-engine contribution.
transaction.scores.ml	ML probability contribution.
transaction.scores.anomaly	Pattern/anomaly contribution.
transaction.explanation.reason	Short reason for admin/user message.
transaction.explanation.rules	Expandable audit details for admin.
transaction.features	Debug/model input, usually admin-only.

Important note:

- If decision === "otp", the transaction is marked pending_otp.
- The current code does not yet implement a transaction OTP verification route, so add that before using OTP as a real payment step.

GET /api/v1/transaction/get-transactions

Purpose: list current user's transactions.

Auth:

- Requires logged-in user.
- Requires accountType === "User".

Success response:

```
{
  "success": true,
  "transactions": [
    {
      "_id": "6660aa000000000000000001",
      "amount": 2500,
      "merchant": "Amazon",
      "category": "shopping",
      "channel": "mobile",
      "deviceId": "dev-android-1",
      "decision": "allow",
      "status": "approved",
      "scores": {
        "ml": 0.03,
        "rule": 0.12,
        "anomaly": 0,
        "risk": 0.063
      },
      "location": {
        "city": "Patna",
        "country": "IN",
        "latitude": 25.5941,
        "longitude": 85.1376
      },
      "createdAt": "2026-05-13T09:20:00.000Z"
    }
  ]
}
```

Frontend use:

- User transaction history table.
- Color rows by decision.
- Show risk badges using scores.risk.

GET /api/v1/transaction/get-all-transactions

Purpose: admin transaction table.

Auth:

- Requires logged-in admin.

Query parameters:

Query	Example	Purpose
page	1	Pagination page. Default 1.
limit	10	Page size. Default 10.
status	blocked	Filter by status: approved, pending_otp, blocked.
decision	block	Filter by decision: allow, otp, block.
minAmount	1000	Minimum transaction amount.
maxAmount	50000	Maximum transaction amount.
search	raushan	Searches user name, user email, merchant, city.

Example request:

```
GET /api/v1/transaction/get-all-transactions?page=1&limit=10&decision=block&search=delhi
Authorization: Bearer <admin-token>
```

Success response:

```
{
  "success": true,
  "total": 42,
  "page": 1,
  "totalPages": 5,
  "transactions": [
    {
      "id": "6660aa00000000000000000000000002",
      "user": {
        "id": "665f1000000000000000000000000001",
        "name": "Raushan Kumar",
        "email": "raushan@example.com",
        "image": "https://api.dicebear.com/5.x/initials/svg?seed=Raushan Kumar",
        "isBlocked": false
      },
      "amount": 95000,
      "merchant": "Unknown merchant",
      "category": "general",
      "channel": "mobile",
      "status": "blocked",
      "decision": "block",
      "scores": {
        "ml": 0.94,
        "rule": 1,
        "anomaly": 1,
        "risk": 1
      },
      "reason": "Too many transactions in short time",
      "statusLabel": "Blocked",
      "deviceId": "new-device-77",
      "location": {
        "city": "Delhi",
        "country": "IN",
        "latitude": 28.6139,
        "longitude": 77.209
      },
      "time": "2026-05-13T09:20:00.000Z",
      "formattedTime": "13/5/2026, 2:50:00 pm",
      "isSuspicious": true
    }
  ]
}
```

Frontend use:

- Admin transaction table.
- Filter by decision/status.
- Click user.id and call /api/v1/admin/users/:userId.

8. Analytics Routes

GET /api/v1/analytics/get-analytics

Purpose: dashboard summary. If logged-in user is admin, it queries all transactions. Otherwise it queries only the logged-in user's transactions.

Auth:

- Any logged-in user.

Success response:

```
{
  "totals": {
    "total": 120,
    "fraud": 8,
    "legit": 112,
    "otp": 14,
    "blocked": 8,
    "avgRisk": 0.318
  },
  "riskTrend": [
    {
      "time": "2026-05-13T09:20:00.000Z",
      "risk": 0.62,
      "amount": 9000,
      "decision": "otp",
      "status": "pending_otp"
    }
  ],
  "fraudByCity": [
    { "city": "Patna", "fraud": 3, "legit": 40 },
    { "city": "Delhi", "fraud": 5, "legit": 22 }
  ],
  "model": {
    "metrics": {
      "accuracy": 0.943,
      "precision": 0.459,
      "recall": 0.737,
      "f1": 0.566,
      "rocAuc": 0.879
    },
    "confusionMatrix": [[679, 33], [10, 28]],
    "rocCurve": [
      { "fpr": 0, "tpr": 0 },
      { "fpr": 0.01, "tpr": 0.42 }
    ]
  }
}
```

Frontend use:

Response field	Visualization
totals.total	Total transaction card.
totals.fraud	Fraud/blocked card.
totals.otp	OTP challenge card.
totals.avgRisk	Average risk gauge.
riskTrend	Line chart: x=time, y=risk; color by decision.
fraudByCity	City-wise fraud bar chart.
model.metrics	ML metric cards.
model.confusionMatrix	2x2 confusion matrix widget.
model.rocCurve	ROC curve line chart.

GET /api/v1/analytics/admin

Purpose: same controller as /get-analytics, but explicitly protected with isAdmin.

Auth:

- Requires admin.

Use this if you want an analytics route that cannot be called by normal users.

9. Admin Routes

GET /api/v1/admin/overview

Purpose: main admin dashboard endpoint.

Auth:

- Requires admin.

Query parameters:

Query	Example	Purpose
from	2026-05-01	Start date filter for createdAt.
to	2026-05-13	End date filter for createdAt.

Example request:

```
GET /api/v1/admin/overview?from=2026-05-01&to=2026-05-13
Authorization: Bearer <admin-token>
```

Success response:

```
{
  "success": true,
  "totals": {
    "totalTransactions": 1200,
    "totalFraud": 86,
    "legit": 1114,
    "approved": 984,
    "otp": 130,
    "blocked": 86,
    "avgRisk": 0.31
  },
  "graphs": {
    "fraudByCity": [
      { "city": "Delhi", "fraud": 18, "totalAmount": 810000, "avgRisk": 0.88 },
      { "city": "Patna", "fraud": 12, "totalAmount": 320000, "avgRisk": 0.81 }
    ],
    "fraudByHour": [
      { "hour": 0, "fraud": 1, "total": 25, "avgRisk": 0.21 },
      { "hour": 1, "fraud": 3, "total": 18, "avgRisk": 0.44 },
      { "hour": 2, "fraud": 11, "total": 36, "avgRisk": 0.62 }
    ],
    "riskDistribution": {
      "block": 86,
      "high": 112,
      "medium": 310,
      "low": 692
    },
    "decisionBreakdown": {
      "allow": 984,
      "otp": 130,
      "block": 86
    },
    "recentRiskTrend": [
      {
        "time": "2026-05-13T09:20:00.000Z",
        "risk": 0.82,
        "rule": 0.7,
        "ml": 0.91,
        "anomaly": 0.8,
        "decision": "block",
        "city": "Delhi"
      }
    ]
  },
  "fraudTiming": {
    "avgFraudHour": 2.42,
    "avgFraudTimeLabel": "02:00"
  },
  "fraudReasons": [
    { "reason": "Impossible travel detected", "count": 22 },
    { "reason": "New device", "count": 16 }
  ],
  "topRiskUsers": [
    {
      "user": {
        "id": "665f10000000000000000001",
        "name": "Raushan Kumar",

```

```

        "email": "raushan@example.com"
    },
    "totalTransactions": 20,
    "fraud": 4,
    "otp": 3,
    "blocked": 4,
    "totalAmount": 420000,
    "avgRisk": 0.72,
    "lastTransactionAt": "2026-05-13T09:20:00.000Z"
  }
],
"recentFrauds": [
  {
    "id": "6660aa00000000000000000002",
    "user": {
      "id": "665f1000000000000000000001",
      "name": "Raushan Kumar",
      "email": "raushan@example.com"
    },
    "amount": 95000,
    "merchant": "Unknown merchant",
    "category": "general",
    "channel": "mobile",
    "city": "Delhi",
    "decision": "block",
    "status": "blocked",
    "scores": {
      "risk": 1,
      "rule": 1,
      "ml": 0.94,
      "anomaly": 1
    },
    "reason": "Too many transactions in short time",
    "createdAt": "2026-05-13T09:20:00.000Z"
  }
]
}

```

Frontend use:

Field	How to show
totals	Top admin KPI cards.
graphs.fraudByCity	City-wise fraud bar chart/map.
graphs.fraudByHour	Fraud timing line/bar chart.
fraudTiming.avgFraudTimeLabel	"Average fraud time" card.
fraudReasons	Reasons table or pie chart.
graphs.riskDistribution	Donut/bar chart for block/high/medium/low.
topRiskUsers	"High-risk users" table. On click, call /api/v1/admin/users/:userId.
recentFrauds	Recent blocked/fraud list.

GET /api/v1/admin/users

Purpose: admin user list with transaction and fraud summary.

Auth:

- Requires admin.

Query parameters:

Query	Example	Purpose
search	raushan	Search by first name, last name, email.
page	1	Page number. Default 1.
limit	20	Page size. Default 20.

Example request:

```

GET /api/v1/admin/users?search=raushan&page=1&limit=10
Authorization: Bearer <admin-token>

```

Success response:

```
{
  "success": true,
  "total": 1,
  "page": 1,
  "totalPages": 1,
  "users": [
    {
      "id": "665f1000000000000000000001",
      "firstName": "Raushan",
      "lastName": "Kumar",
      "name": "Raushan Kumar",
      "email": "raushan@example.com",
      "accountType": "User",
      "isBlocked": false,
      "image": "https://api.dicebear.com/5.x/initials/svg?seed=Raushan Kumar",
      "behavior": {
        "avgAmount": 2200,
        "stdAmount": 850,
        "maxAmount": 6000,
        "transactionCount": 12
      },
      "createdAt": "2026-05-01T10:00:00.000Z",
      "totalTransactions": 20,
      "totalAmount": 420000,
      "fraud": 4,
      "avgRisk": 0.72,
      "lastTransactionAt": "2026-05-13T09:20:00.000Z"
    }
  ]
}
```

Frontend use:

- Admin users table.
- Show fraud, avgRisk, lastTransactionAt.
- On user row click, call /api/v1/admin/users/:userId.

GET /api/v1/admin/users/:userId

Purpose: clicked-user detail page.

Auth:

- Requires admin.

Example request:

```
GET /api/v1/admin/users/665f1000000000000000000001
Authorization: Bearer <admin-token>
```

Success response:

```
{
  "success": true,
  "user": {
    "id": "665f1000000000000000000001",
    "firstName": "Raushan",
    "lastName": "Kumar",
    "name": "Raushan Kumar",
    "email": "raushan@example.com",
    "accountType": "User",
    "isBlocked": false,
    "behavior": {
      "avgAmount": 2200,
      "stdAmount": 850,
      "maxAmount": 6000,
      "transactionCount": 12,
      "commonDevices": ["dev-android-1"],
      "commonLocations": ["Patna"]
    }
  },
  "profile": {
    "gender": "Male",
    "address": {
      "city": "Patna",
      "state": "Bihar",
      "country": "India"
    }
  },
  "summary": {
    "totalTransactions": 20,
    "fraud": 4,
  }
}
```

```

    "legit": 16,
    "otp": 3,
    "avgRisk": 0.72
  },
  "graphs": {
    "cityBreakdown": [
      { "city": "Delhi", "total": 5, "fraud": 3 },
      { "city": "Patna", "total": 15, "fraud": 1 }
    ],
    "riskTrend": [
      {
        "time": "2026-05-13T09:20:00.000Z",
        "risk": 0.82,
        "rule": 0.7,
        "ml": 0.91,
        "anomaly": 0.8,
        "decision": "block"
      }
    ]
  },
  "fraudReasons": [
    { "reason": "Impossible travel detected", "count": 2 },
    { "reason": "New device", "count": 1 }
  ],
  "transactions": [
    {
      "_id": "6660aa000000000000000002",
      "amount": 95000,
      "decision": "block",
      "status": "blocked",
      "scores": {
        "risk": 1,
        "rule": 1,
        "ml": 0.94,
        "anomaly": 1
      }
    }
  ]
}

```

Frontend use:

- User detail header from user and profile.
- Summary cards from summary.
- City chart from graphs.cityBreakdown.
- Risk history chart from graphs.riskTrend.
- Reasons table from fraudReasons.
- Transaction audit table from transactions.

GET /api/v1/admin/scoring-guide

Purpose: gives frontend a stable explanation of scoring levels and formulas.

Auth:

- Requires admin.

Success response:

```

{
  "success": true,
  "ruleSeverityToNumber": {
    "critical": 1,
    "high": "0.7 - 0.9 depending on rule",
    "medium": "0.3 - 0.6 depending on rule",
    "low": "0.1 - 0.2 depending on rule"
  },
  "graphBands": {
    "block": "risk >= 0.8",
    "high": "0.6 <= risk < 0.8",
    "medium": "0.35 <= risk < 0.6",
    "low": "risk < 0.35"
  },
  "finalRiskFormula": {
    "withMl": "0.45 * ruleScore + 0.25 * anomalyScore + 0.30 * mlFraudProbability",
    "withoutMl": "0.50 * ruleScore + 0.50 * anomalyScore",
    "hardRule": "critical or multiple high-risk rules can block directly; numeric risk is still stored as 1.0 for graphing and review"
  },
  "mlRole": [

```



```

    "ML does not override critical rule blocks.",
    "ML adds probability for borderline cases where rules are not critical.",
    "For direct blocks, ML is still useful as audit evidence, monitoring signal, and retraining
feedback."
  ],
  "metricNote": {
    "tpr": "TPR/recall = true fraud blocked / all known fraud in the labeled test set.",
    "fpr": "FPR = legitimate transactions wrongly blocked / all known legitimate transactions in the
labeled test set.",
    "productionReality": "A live blocked transaction is not automatically known fraud. You need later
labels from user complaints, bank chargebacks, manual review, or confirmed safe OTP retries to calculate
production FPR/TPR."
  }
}

```

Frontend use:

- Show legend/tooltips in admin graphs.
- Keep risk color logic consistent with backend.

10. Flask ML Service Routes

Base URL:

`http://localhost:5000`

Route summary:

Method	Route	Purpose
GET	/	Service info and endpoint list.
GET	/health	Health and model-loaded status.
GET	/model/info	Model type, dataset metadata, feature columns, metrics.
GET	/metrics	Dashboard-friendly ML metrics.
POST	/predict	Predict one transaction feature row.
POST	/predict/batch	Predict many transaction feature rows.

GET /health

Response:

```

{
  "status": "ok",
  "model_loaded": true
}

```

GET /model/info

Response:

```

{
  "model_type": "RandomForestClassifier",
  "dataset": {
    "name": "Credit Card Fraud Detection with Synthetic Behavioral Features",
    "format": "Kaggle creditcard.csv labels + engineered rule-engine-style features",
    "source_url": "https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv",
    "raw_rows": 284807,
    "enhanced_dataset_path": "E:\\CAPSTONE\\ml-service\\data\\enhanced_fraud_features.csv",
    "note": "Behavioral columns are synthetic because Kaggle hides user, device, merchant, and location
data."
  },
  "feature_columns": [
    "amount",
    "hour_of_day",
    "velocity_10m",
    "velocity_1h",
    "time_gap_minutes",
    "geo_distance_km",
    "device_changed",
    "amount_deviation",
    "night_transaction",
    "known_device",
    "known_location"
  ],
}

```

```

    "metrics": {
      "roc_auc": 0.98,
      "confusion_matrix": [[56850, 14], [20, 78]],
      "test_rows": 56962,
      "fraud_rate": 0.0017
    }
  }
}

```

GET /metrics

Purpose: analytics-friendly metrics endpoint used by Express analytics.

Response:

```

{
  "accuracy": 0.9994,
  "precision": 0.8478,
  "recall": 0.7959,
  "f1": 0.8211,
  "roc_auc": 0.982,
  "confusion_matrix": [[56850, 14], [20, 78]],
  "roc_curve": [
    { "fpr": 0.0, "tpr": 0.0, "threshold": "Infinity" },
    { "fpr": 0.001, "tpr": 0.78, "threshold": 0.62 }
  ],
  "test_rows": 56962,
  "fraud_rate": 0.0017,
  "threshold": 0.5
}

```

Note:

- Actual values depend on your trained artifact.
- If you changed train.py, run python train.py again so roc_curve is saved into the model artifact.

POST /predict

Purpose: predict fraud probability for one feature row.

Request:

```

{
  "features": {
    "amount": 1250,
    "hour_of_day": 16,
    "velocity_10m": 5,
    "velocity_1h": 8,
    "time_gap_minutes": 0.5,
    "geo_distance_km": 720,
    "device_changed": 1,
    "amount_deviation": 3.4,
    "night_transaction": 0,
    "known_device": 0,
    "known_location": 0
  }
}

```

Success response:

```

{
  "is_fraud": true,
  "fraud_probability": 0.86,
  "threshold": 0.5
}

```

You can also send features directly without nesting:

```

{
  "amount": 1250,
  "hour_of_day": 16,
  "velocity_10m": 5,
  "velocity_1h": 8,
  "time_gap_minutes": 0.5,
  "geo_distance_km": 720,
  "device_changed": 1,
  "amount_deviation": 3.4,
  "night_transaction": 0,
  "known_device": 0,
  "known_location": 0
}

```

POST /predict/batch

Purpose: predict many rows at once.

Request:

```
{
  "transactions": [
    {
      "amount": 1250,
      "hour_of_day": 16,
      "velocity_10m": 0,
      "velocity_1h": 1,
      "time_gap_minutes": 120,
      "geo_distance_km": 0,
      "device_changed": 0,
      "amount_deviation": 0.2,
      "night_transaction": 0,
      "known_device": 1,
      "known_location": 1
    },
    {
      "amount": 90000,
      "hour_of_day": 2,
      "velocity_10m": 8,
      "velocity_1h": 11,
      "time_gap_minutes": 1,
      "geo_distance_km": 1500,
      "device_changed": 1,
      "amount_deviation": 6.1,
      "night_transaction": 1,
      "known_device": 0,
      "known_location": 0
    }
  ]
}
```

Response:

```
{
  "threshold": 0.5,
  "results": [
    { "index": 0, "is_fraud": false, "fraud_probability": 0.04 },
    { "index": 1, "is_fraud": true, "fraud_probability": 0.93 }
  ]
}
```

11. Data Models

User

Important fields:

Field	Meaning
firstName, lastName, email, password	Account identity and login.
accountType	"User" or "Admin".
isBlocked	If true, auth rejects user.
behavior.avgAmount	User's running average transaction amount.
behavior.stdAmount	Running standard deviation of amount.
behavior.maxAmount	Highest approved/non-blocked transaction amount.
behavior.transactionCount	Count of accepted behavior-profile transactions.
behavior.commonDevices	Recent known devices.
behavior.commonLocations	Recent known cities.
behavior.lastTransactionAt	Last accepted transaction time.
behavior.lastLatitude, lastLongitude	Last accepted transaction location.

Transaction

Important fields:

Field	Meaning
user	User who made the transaction.
amount, merchant, category, channel, accountId	Transaction details.
deviceId	Device used.
location	City, country, latitude, longitude.
features	Feature set used by rules and ML.
scores.ml	ML fraud probability.
scores.rule	Weighted rule score.
scores.anomaly	Anomaly score.
scores.risk	Final risk score used for decision/graphs.
scores.graph	Same graph-friendly score.
adaptivePolicy	Dynamic thresholds based on user history.
decision	allow, otp, block.
status	approved, pending_otp, blocked.
explanation	Human-readable reason and rule list.

12. Feature Engineering Logic

When a user creates a transaction, buildFeatures creates this feature object:

Feature	Backend calculation	Meaning
amount	Number (payload.amount)	Transaction amount.
hour_of_day	Current server hour	Timing feature.
velocity_10m	Count of user's transactions in last 10 minutes	Burst/frequency signal.
velocity_1h	Count of user's transactions in last 1 hour	Hourly frequency signal.
time_gap_minutes	Minutes since user's last transaction; 9999 if none	Very small gap can be risky.
geo_distance_km	Distance from last transaction location to current location	Used for impossible travel.
device_changed	1 if device differs from last transaction	Device anomaly.
amount_deviation	$\text{abs}(\text{amount} - \text{avgAmount}) / \text{stdAmount}$	Amount anomaly compared to user pattern.
night_transaction	1 if hour < 6 or > 22	Night risk signal.
known_device	1 if device exists in commonDevices	Known device lowers risk.
known_location	1 if city exists in commonLocations	Known location lowers risk.

After a transaction is not blocked, updateBehaviorProfile updates the user's behavior:

- running avgAmount
- running stdAmount
- maxAmount
- transactionCount
- recent commonDevices
- recent commonLocations
- last transaction time and location

Blocked transactions do not update behavior profile. This prevents fraudulent activity from becoming the user's "normal" pattern.

13. Rule-Based Fraud Logic

The rule engine evaluates five rule groups:

Rule	Low	Medium	High	Critical
Velocity	Normal movement / no history	Speed > 100 km/h	Speed > 200 or > 600 km/h	Speed > 900 km/h or zero time gap
Frequency	Normal frequency	velocity_10m >= 3	velocity_10m >= 5 or velocity_1h > 15	velocity_10m >= 7
Amount	Slightly above average	Amount > avg * 2	Amount > max * 1.5	No separate critical amount rule
Device	Known device	No medium device rule	Missing/new device	No separate critical device rule
Location	Known city	No history/new city	No high location rule	No separate critical location rule

Rule score examples:

```
critical -> usually 1.0
high     -> usually 0.7 to 0.9
medium   -> usually 0.3 to 0.6
low      -> usually 0.1 to 0.2
```

Hard Rule Decisions

The system directly blocks if:

```
impossible travel is detected
OR velocity_10m >= 7
OR any rule is critical
OR two or more rules are high
```

The system asks for OTP if:

```
exactly one rule is high
```

Why hard rules are useful:

- Some fraud patterns should not wait for ML.
- Example: impossible travel or burst transactions are strong enough to block immediately.

Weighted Rule Score

If no hard rule decides the transaction, the system calculates a weighted rule score:

```
weights = {
  velocity: 0.30,
  frequency: 0.20,
  amount: 0.20,
  location: 0.15,
  device: 0.15
}
```

Formula:

```
ruleScore = sum(rule.score * ruleWeight)
ruleScore = min(ruleScore, 1)
```

14. Anomaly Score Logic

The anomaly score is separate from rule score. It adds risk for abnormal patterns:

```
amount_deviation > 3    -> +0.50
amount_deviation > 2    -> +0.30
amount_deviation > 1    -> +0.10

geo_distance_km > 1000  -> +0.30
geo_distance_km > 500   -> +0.20
geo_distance_km > 100   -> +0.10

velocity_10m >= 5       -> +0.30
velocity_10m >= 3       -> +0.20
velocity_1h > 10        -> +0.20
```

```
night_transaction == 1    -> +0.05
known_device == 0         -> +0.10
known_location == 0       -> +0.10
```

Final:

```
anomalyScore = min(total, 1)
```

15. ML + Rule Combination Logic

The current branch combines rule-based and ML scoring in `riskService.js`.

ML Score

The Express server calls:

```
POST http://localhost:5000/predict
```

with:

```
{
  "features": {
    "amount": 2500,
    "hour_of_day": 14,
    "velocity_10m": 0,
    "velocity_1h": 1,
    "time_gap_minutes": 90.2,
    "geo_distance_km": 0,
    "device_changed": 0,
    "amount_deviation": 0.42,
    "night_transaction": 0,
    "known_device": 1,
    "known_location": 1
  }
}
```

The ML service returns:

```
{
  "is_fraud": false,
  "fraud_probability": 0.03,
  "threshold": 0.5
}
```

Then Express stores:

```
{
  "scores": {
    "ml": 0.03
  }
}
```

If Hard Rule Triggers

Hard rules still make the final decision. ML does not override them.

For example:

```
critical rule -> block
two high rules -> block
one high rule -> otp
```

But ML is still stored for:

- admin graphing
- audit explanation
- monitoring false positives later
- future retraining labels

Hard block numeric scoring:

```

block hard rule -> hardScore = 1.0
otp hard rule   -> hardScore = 0.75

risk = max(hardScore, ruleScore, anomalyScore, mlScore)

```

If No Hard Rule Triggers

If ML is available:

```

finalRisk = 0.45 * ruleScore
           + 0.25 * anomalyScore
           + 0.30 * mlFraudProbability

```

If ML is unavailable:

```

finalRisk = 0.50 * ruleScore
           + 0.50 * anomalyScore

```

This fallback is important because the transaction system can still work even if the Python ML service is down.

16. Adaptive Decision Policy

The system adjusts thresholds using user transaction history:

```

transactionCount > 50 -> confidence = 1.0
transactionCount > 20 -> confidence = 0.8
transactionCount > 5  -> confidence = 0.5
otherwise          -> confidence = 0.2

```

Thresholds:

```

otpThreshold  = 0.6 - confidence * 0.2
blockThreshold = 0.8 - confidence * 0.2

```

Decision:

```

if risk >= blockThreshold -> decision = block, status = blocked
if risk >= otpThreshold   -> decision = otp, status = pending_otp
else                     -> decision = allow, status = approved

```

Meaning:

- More history means more confidence.
- More confidence means stricter thresholds.
- A long-time user's unusual behavior is more suspicious than a new user's unknown behavior.

17. ML Training and Dataset Logic

The ML model is trained in `ml-service/train.py`.

Original Kaggle-like columns:

```
Time, Amount, V1...V28, Class
```

Your model uses:

```

amount
hour_of_day
velocity_10m
velocity_1h
time_gap_minutes
geo_distance_km
device_changed
amount_deviation
night_transaction
known_device
known_location

```

Target:

is_fraud

Important note:

The Kaggle dataset does not contain real `userId`, `deviceId`, `city`, `latitude`, `longitude`, `merchant`, or `user history`. Therefore, `train.py` creates synthetic behavior features that imitate your rule engine.

Training steps:

1. Download/read `creditcard.csv`.
2. Generate synthetic behavior features.
3. Split data into 80% train and 20% test using `stratify`.
4. Scale continuous columns.
5. Train `RandomForestClassifier`.
6. Predict probabilities on the test set.
7. Calculate classification report, confusion matrix, ROC AUC, and ROC curve points.
8. Save model artifact to `ml-service/artifacts/fraud_random_forest.joblib`.

18. ROC, TPR, FPR, Precision, Recall

Confusion Matrix Terms

Term	Meaning
TP	Actual fraud and predicted fraud.
FP	Actual legitimate but predicted fraud.
TN	Actual legitimate and predicted legitimate.
FN	Actual fraud but predicted legitimate.

TPR / Recall

Formula:

$$\text{TPR} = \text{TP} / (\text{TP} + \text{FN})$$

Meaning:

- Out of all actual fraud transactions, how many did the model catch?
- Higher TPR means fewer missed frauds.

FPR

Formula:

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

Meaning:

- Out of all actual legitimate transactions, how many were wrongly flagged as fraud?
- Lower FPR means fewer innocent users are blocked.

Precision

Formula:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Meaning:

- Out of all transactions predicted fraud, how many were truly fraud?

ROC Curve

The ML model outputs a probability:

```
{ "fraud_probability": 0.86 }
```

A threshold converts it into a decision:

```
if fraud_probability >= threshold -> fraud  
else -> not fraud
```

ROC tests many thresholds and plots:

```
x-axis = FPR  
y-axis = TPR
```

ROC AUC

ROC AUC is the area under the ROC curve.

Meaning:

- 1.0 is perfect ranking.
- 0.5 is random guessing.
- Higher AUC means the model separates fraud and legitimate rows better.

Is Current FPR/TPR Production Data?

No. Current FPR/TPR is calculated on the held-out test split from training:

```
X_train, X_test, y_train, y_test = train_test_split(...)  
fpr, tpr, roc_thresholds = roc_curve(y_test, probabilities)
```

That means:

- Training dataset creates model.
- Test split evaluates model.
- Live production transactions are not automatically labeled.

Production FPR/TPR needs later labels:

- manual fraud review
- chargeback confirmation
- user complaint
- confirmed legitimate OTP retry
- bank dispute result

Until you have those labels, production dashboard can show risk scores and decisions, but not true live FPR/TPR.

19. How Frontend Should Use Response Data

User Dashboard

Use:

API	Use
/api/v1/profile/me	Current user card/profile.
/api/v1/transaction/get-transactions	Transaction history table.
/api/v1/analytics/get-analytics	User risk trend and personal fraud summary.

Recommended UI:

- Recent transactions table.
 - Risk chart from riskTrend.
 - Cards for total, blocked, OTP, average risk.
 - Status badges:
- allow/approved -> green - otp/pending_otp -> yellow - block/blocked -> red

Admin Dashboard

Use:

API	Use
/api/v1/admin/overview	Main admin dashboard.
/api/v1/admin/users	User table.
/api/v1/admin/users/:userId	Clicked user detail page.
/api/v1/transaction/get-all-transactions	Admin transaction table.
/api/v1/admin/scoring-guide	Tooltips/legend/scoring explanation.

Recommended graphs:

Graph	Data
City-wise fraud bar chart	graphs.fraudByCity
Fraud by hour chart	graphs.fraudByHour
Average fraud time card	fraudTiming.avgFraudTimeLabel
Risk distribution donut	graphs.riskDistribution
Decision breakdown	graphs.decisionBreakdown
Risk trend line	graphs.recentRiskTrend
Fraud reasons table	fraudReasons
Top risky users table	topRiskUsers

Click User Flow

Recommended flow:

1. Admin opens /admin dashboard.
2. Frontend calls GET /api/v1/admin/overview.
3. Render topRiskUsers.
4. Admin clicks a user row.
5. Frontend reads user.id.
6. Frontend calls GET /api/v1/admin/users/:userId.
7. Render user detail, profile, transactions, city chart, risk trend, reasons.

20. Important Implementation Notes

Item	Current status	Suggestion
Admin routes	Present and mounted at /api/v1/admin.	Use these for admin dashboard.
ML integration	Present in riskService.js.	Keep ML advisory for hard rules; include in graphs.
Hard-rule graph score	Present: hard block stores numeric scores.	Good for graphing direct blocks.
Transaction OTP	Decision/status exist, but OTP verification route is not active.	Add OTP generation and verify route before production.
ML metrics	/metrics exists in Flask app.	Retrain model after changes so artifact contains roc_curve.
Analytics route	Existing /analytics/get-analytics remains useful.	Admin dashboard should prefer /admin/overview.

21. Quick Test Order

Start Express:

```
cd E:\CAPSTONE\server
npm install
npm start
```

Start ML:

```
cd E:\CAPSTONE\ml-service
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python train.py
python app.py
```

Test flow:

1. POST /api/v1/auth/sendotp
2. POST /api/v1/auth/signup
3. POST /api/v1/auth/login
4. POST /api/v1/profile/updateprofile
5. POST /api/v1/transaction/create-transaction
6. GET /api/v1/transaction/get-transactions
7. GET /api/v1/analytics/get-analytics
8. GET /api/v1/admin/overview with admin token
9. GET /api/v1/admin/users with admin token
10. GET /api/v1/admin/users/:userId with admin token

Minimum .env values for Express:

```
PORT=4000
MONGODB_URL=<mongodb_connection_string>
JWT_SECRET=<jwt_secret>
GEOCODING_API=<opencage_api_key>
ML_SERVICE_URL=http://localhost:5000
MAIL_HOST=<smtp_host>
MAIL_USER=<smtp_user>
MAIL_PASS=<smtp_password>
CLOUD_NAME=<cloudinary_cloud_name>
API_KEY=<cloudinary_api_key>
API_SECRET=<cloudinary_api_secret>
```